

OS_Project3: Add mmap & munmap to xv6

Class: HYU-ELE3021

2019069270 유규태

Step 1: Modify sys_mmap(void) (kernel/sysfile.c)

p=myproc() 한 후 used 값이 0인 slot 찾은 후, 겹치지 않는 구간으로 addr 값 설정

test: after make qemu and mmaptest ->

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
test basic mmap
usertrap(): unexpected scause 0xd pid=3
              sepc=0x10a stval=0x3fffffc000
$
```

Fail after first mmap success. (first access page fault, b/c mmapfault() <- return -1;)

Step 2: Modify mmapfault(uint64 va)

1. Before allocating physical page, find vma including va

(이미 매핑된 페이지이면 return -1 처리)

2. Mem(커널주소)에 physical page 할당 (memset() 함수 이용)
3. 파일에서 해당 페이지의 데이터 읽기

Kernel/File.c의 fileread() 함수 로직을 참고해 readi() 함수 사용함. 이 때 둘째 파라미터를 1이 아닌 0 사용(mmapfault() 함수에서 mem은 커널주소로 사용되는 중) 추가로 File offset 변수 값 설정 후 readi() 함수 및 위아래로 락을 걸고 풀

4. PTE 권한 변환

Ppt 내용 참고해서 perm 값 설정.

5. 물리 페이지를 va에 매핑

mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm) 후 return mem;

```

hart 1 starting
init: starting sh
$ mmaptest
test basic mmap
usertrap(): unexpected scause 0xd pid=3
          sepc=0x10a stval=0x3fffffc000
panic: freewalk: leaf
OEMU: Terminated

```

Usertrap()이 다시 호출되는 문제 해결: trap.c에서 실패로 간주해 프로세스가 죽고 mmap 페이지 미정리로 인해 발생한 것을 알게 됨. Mmapfault return 값을 음수가 아닌 0으로 리턴한다(trap.c : mmapfault(stval)==0: //lazy fault handled.)

이후 mmaptest:

```

hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
test basic mmap
mmaptest: test basic mmap failed: munmap, pid=3
panic: freewalk: leaf

```

이제 유저트랩 멘트가 없는 것을 확인 후 step3 진행함.

Step3: Modify munmap(void)

1. Addr 포함하는 vma 찾고, middle hole이 생기는 조건문 추가(return -1)

For문에서 Addr == p->vmass[i].addr 또는 addr+length==p->vmass[i].addr+length일 때 앞 또는 뒤에서 자르기 가능(나머지는 중간에 위치하는 경우임)

2. If a page never faulted before, skip: 매핑된 페이지가 없으면 continue

If walkaddr(p->pagetable, va)==0 continue;

3. MAP_SHARED: write back data to the file

Begin_op()/end_op() 호출 이유 - 커널 패닉을 막기 위해서이다 (ppt writei() call 참고함)

락앤언락 사이에 writei() 함수로 write back

4. PTE clear, free physical page (uvmunmap() 호출)
5. Vma 범위 조정(앞에서 자른 경우 오프셋을 그만큼 앞으로 당김)
 - ➔ If addr == vma->addr -> vma 오프셋과 주소를 length만큼 더함
 - ➔ 별개로 vma_length += length
6. Close file and free vma slot if whole vma gone (fileclose() 호출)

Sys_mmap(), mmapfault(), munmap() 수정 후 mmaptest, usertests -q 실행결과:

```
kyutaeg@DESKTOP-KU1KG90:~/project3-braeden-hue$ make qemu
$ mmaptest
test basic mmap
test basic mmap: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
usertrap(): unexpected scause 0xd pid=5
sepc=0x10a stval=0x3fffffc000
mmaptest: test fork failed: child exit, pid=4
```

```
$ usertests -q
usertests starting
test copyin: PASS
test copyout: PASS
test copyinstr1: PASS
```

(중간에 유저트랩 여러 번 발생)

```
usertrap(): unexpected scause 0xf pid=6643
sepc=0x4568 stval=0x3e014000
usertrap(): unexpected scause 0xf pid=6644
sepc=0x4568 stval=0x3f014000
PASS
test lazy_copy: PASS
test lazy_sbrk: PASS
PASS ALL TESTS
```

Test fork만 수정하면 되는 것을 확인했다. 또 step3 까지의 구현이 기존 기능을 오염시키지 않은 듯하다(pass all tests 확인)

Step4: Hook kfork and kexit

Kfork(): child must see same mappings as the parent

Open file descriptor 복사 직후 실행

➔ Np->vmass[i]에 p->vmass[i] 복사, filedup(np->vmass[i].f) (물리 페이지는 공유 x)

Kexit(): write back 실행(pagetable 해제 전, open file 닫힌 후)

➔ Vma->f는 filedup() 때문에 별도로 close 필요함

```
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
test munmap prevents access
usertrap(): unexpected scause 0xd pid=5
sepc=0xa6e stval=0x3fffffd000
usertrap(): unexpected scause 0xd pid=6
sepc=0xae8 stval=0x3fffffc000
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): unexpected scause 0xf pid=7
sepc=0xc3e stval=0x3fffffc000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded
```

```
usertrap(): unexpected scause 0xf pid=6642
sepc=0x4568 stval=0x3b014000
usertrap(): unexpected scause 0xf pid=6643
sepc=0x4568 stval=0x3c014000
usertrap(): unexpected scause 0xf pid=6644
sepc=0x4568 stval=0x3d014000
usertrap(): unexpected scause 0xf pid=6645
sepc=0x4568 stval=0x3e014000
usertrap(): unexpected scause 0xf pid=6646
sepc=0x4568 stval=0x3f014000
PASS
test lazy_copy: PASS
test lazy_sbrk: PASS
PASS ALL TESTS
```

왼쪽은 mmaptest, 오른쪽은 usertests -q 실행 결과의 일부분(뒷부분)이다. 최종 확인 결과 모두 pass 또는 succeed 나온 것 확인